

Week 5 · 실습 워크북: 코드를 모듈로 나누고, envs/prod 에서 조립합니다

📌 참고

이 문서는 실습 파트(45분)입니다. 개념은 같은 `lecture/` 폴더의 **개념워크북** 을 먼저 보세요. 여기서는 번호만 참조합니다. **[실습]** 은 직접 입력하고, **[확인]** 은 눈으로 대조하고, **[함정]** 은 미리 피하고, **[관찰 🕒]** 은 빈칸을 채웁니다. **(심화 🌟)** 는 먼저 끝낸 사람만 합니다.

작업 폴더는 `practice/` 입니다. `solution/` 은 막혔을 때 답을 보는 곳입니다. 처음부터 끝까지 순서대로 진행하면 완료됩니다.

오늘 만들 것

리소스 9개입니다. week4와 종류도 개수도 같습니다. 달라지는 것은 그 9개가 어느 모듈 안에 들어가고, state 주소가 어떻게 바뀌는지입니다.

폴더	무엇을 하는 곳	state 는 어디에
<code>practice/modules/network/</code>	VPC + IGW + 서브넷 2 + 라우트 테이블 + 연결 2 를 정의만 합니다	없습니다
<code>practice/modules/compute/</code>	시큐리티 그룹 + EC2 를 정의만 합니다	없습니다
<code>practice/envs/prod/</code>	두 모듈을 조립해 실제로 만듭니다. 리소스 9개	week3 의 S3 버킷 (<code>week05/prod/...</code>)

`terraform` 명령을 치는 곳은 `practice/envs/prod` 한 곳뿐입니다. `modules/` 안에서는 `init` 도 `plan` 도 실행하지 않습니다. 그 폴더에는 `provider` 도 `backend` 도 없습니다.

🚫 절대 금지

비용. `envs/prod` 를 apply 하면 시간당 \$0.0190이 발생합니다. destroy 를 실행하지 않고 한 달을 두면 약 \$13.87입니다. Block C는 필수입니다. `module` 블록에 `for_each` 를 걸어 여러 벌 만들어보지 마세요. compute 모듈이 두 벌이면 EC2 가 2대가 됩니다. 개념워크북 14번.

올리면 안 되는 값. 본인 공인 IP · AWS 계정번호 12자리 · 액세스 키 · `terraform.tfvars` · `terraform.tfstate` · `.pem` 파일. 마스킹 명령은 C-3에 있습니다.

🚫 절대 금지

공인 IP 는 화면에서는 숨겨지고 저장소에서는 숨겨지지 않습니다. `envs/prod/variables.tf` 와 `modules/compute/variables.tf` 의 `my_ip` 에 `sensitive = true` 가 붙어 있어서, plan 출력의 `ingress` 블록이 (`sensitive value`) 로 나옵니다. week4와 같습니다.

숨겨지는 것은 CLI 출력뿐입니다. `terraform.tfvars` 에는 평문으로 저장되고 state 에도 평문으로 기록됩니다. 그래서 그 두 파일은 커밋 금지입니다. 무엇이 숨겨지고 무엇이 숨겨지지 않는지는 B-6에서 직접 확인합니다.

! 꼭 기억

오늘 폴더가 셋입니다. `modules/network` · `modules/compute` · `envs/prod`. 파일을 여는 곳은 셋이고 명령을 치는 곳은 하나입니다. 이 문서의 코드 블록 첫 줄에 붙은 `# pwd == ...` 주석은 그 블록의 명령을 모두 실행한 뒤에 있어야 하는 위치입니다. 블록 안에 `cd` 가 있으면 시작 위치는 이전 블록의 값입니다. 각 블록 끝에 `pwd` 로 대조하세요.

파일	오늘 손대나
<code>envs/prod/versions.tf</code> · <code>providers.tf</code> · <code>variables.tf</code> · <code>locals.tf</code>	읽기만
<code>envs/prod/backend.tf</code>	TODO ① week3 버킷 이름을 직접 입력
<code>modules/network/variables.tf</code> · <code>outputs.tf</code>	읽기만. 입출력 계약입니다
<code>modules/network/main.tf</code>	TODO ② 리소스 5개
<code>modules/compute/variables.tf</code> · <code>outputs.tf</code>	읽기만. 입출력 계약입니다
<code>modules/compute/main.tf</code>	TODO ④ 데이터 소스 1개 + 리소스 2개
<code>envs/prod/main.tf</code>	TODO ③⑤ <code>module</code> 블록 2개. A-4 에서 하나, B-2 에서 하나
<code>envs/prod/outputs.tf</code>	TODO ⑥ 모듈 output 재노출
<code>envs/prod/example.tfvars</code>	<code>terraform.tfvars</code> 로 복사해 값 채우기

계약 파일을 읽기만 하는 것이 오늘의 실습 설계입니다. 모듈을 만들 때 먼저 정하는 것이 "무엇을 받아 무엇을 내줄 것인가"이고, 그 안을 채우는 일이 `main.tf` 입니다. 개념워크북 7번.

⚠ 주의

`practice/` 의 `outputs.tf` 들은 아직 존재하지 않는 리소스를 참조하고 있습니다.
`modules/network/outputs.tf` 는 `aws_vpc.this` 를, `modules/compute/outputs.tf` 는 `aws_instance.web` 을 가리킵니다. 그래서 `main.tf` 를 채우기 전에는 `validate` 가 일부러 실패합니다. 계약이 먼저 있고 구현이 나중에 오기 때문입니다.

오늘의 지도: 폴더 하나가 셋이 되고, 값이 문을 통해 건너갑니다

week1의 지도는 명령의 흐름, week2는 리소스 사이의 참조, week3은 state 파일의 위치, week4는 값이 어디에 정의되는지였습니다. 오늘의 지도는 코드가 어느 폴더에 있는지와, 폴더 경계를 값이 어떻게 넘어가는지를 보는 것입니다.

① BEFORE: 오늘 시작 시점 (week4 가 끝난 상태)

```
practice/
  versions.tf providers.tf backend.tf
  variables.tf locals.tf outputs.tf
  network.tf    VPC · IGW · 서브넷 x2 · 라우트 테이블 · 연결 x2
  compute.tf    시큐리티 그룹 · EC2
```

폴더가 하나입니다.

compute.tf 의 aws_instance.web 이 network.tf 의 aws_subnet.public["a"] 를 아무 절차 없이 바로 참조합니다. 파일 이름으로 나뉘어 있을 뿐이고 Terraform 은 이 폴더를 하나의 통합 단위로 읽습니다.

dev 를 만들려면 이 폴더를 통째로 복사해야 하고, 복사한 순간 리소스 정의가 두 벌이 됩니다.

② AFTER: 오늘 끝날 때

```
practice/
  modules/
    network/  variables.tf  main.tf  outputs.tf
              (받는 것)    (처리)    (내주는 것)
    compute/  variables.tf  main.tf  outputs.tf
  envs/
    prod/     versions.tf  providers.tf  backend.tf
              variables.tf  locals.tf
              main.tf      module 블록 2개. 조립만 합니다
              outputs.tf
```

terraform 명령은 envs/prod 에서만 실행합니다.
modules/ 는 코드 정의일 뿐이고 state 를 갖지 않습니다.
envs/dev 를 추가할 때 복사하는 것은 루트 모듈뿐이고, 리소스 정의는 한 벌로 유지됩니다.

③ 값이 건너가는 길

envs/prod/variables.tf		modules/network/		AWS
var.vpc_cidr	┌───┐	var.vpc_cidr	───>	aws_vpc.this
var.subnet_cidrs	┌───┐	var.subnet_cidrs	───>	aws_subnet.public["...-2a"]
		var.name_prefix		["...-2c"]
envs/prod/locals.tf		var.tags	───>	aws_internet_gateway.this
local.name_prefix	└───┘			aws_route_table.public
local.common_tags	└───┘			aws_route_table_association x2
		output "vpc_id"		
		output "subnet_ids"		
		output "public_subnet_id"		
		└───┘		
				envs/prod/main.tf 의 두 줄에서만 이어집니다
		v		
		modules/compute/		

```

var.vpc_id      —> aws_security_group.web
var.subnet_id   —> aws_instance.web
var.my_ip
var.name_prefix
var.tags

output "instance_id"
output "public_ip"

```

가운데 세로선을 보세요. network 가 내보낸 output 을 compute 의 변수로 전달하는 지점이고, 그 지점은 `envs/prod/main.tf` 두 줄뿐입니다. 두 모듈은 서로를 알지 못합니다.

④ 주소 지도: 접두사가 붙습니다

week4 의 주소

```

aws_vpc.main
aws_subnet.public["a"]
aws_subnet.public["c"]
aws_security_group.web
aws_instance.web
data.aws_ami.al2023

```

오늘의 주소

```

module.network.aws_vpc.this
module.network.aws_subnet.public["ap-northeast-2a"]
module.network.aws_subnet.public["ap-northeast-2c"]
module.compute.aws_security_group.web
module.compute.aws_instance.web
module.compute.data.aws_ami.al2023
~~~~~

```

모듈 경로 접두사. module 블록의 라벨입니다.

두 군데가 달라졌습니다. 앞에 module.<라벨>. 이 붙었고,
서브넷의 key 가 "a" 에서 "ap-northeast-2a" 로 바뀌었습니다.
뒤의 변화는 subnet_cidrs 의 맵 구조가 달라진 것이고 모듈화와는 별개입니다.

스텝 ↔ 지도 대응표

스텝	지도에서 하는 일
A-2	②의 <code>envs/prod/backend.tf</code> 를 week3 버킷에 연결합니다
A-3	②의 <code>modules/network/main.tf</code> 칸을 채웁니다
A-4	②의 <code>modules/network</code> 가 Terraform 에 등록되는 것을 봅니다
A-5	③의 세로선이 유일한 통로라는 것을 오류로 확인합니다
B-1	②의 <code>modules/compute/main.tf</code> 칸을 채웁니다
B-2	③의 화살표 전체를 잊습니다
B-3	③의 오른쪽 output 을 루트 밖으로 다시 냅니다
B-5	④를 <code>state list</code> 로 확인합니다
B-6	③의 <code>public_subnet_id</code> 대신 <code>subnet_ids</code> 로 고칩니다

스텝	지도에서 하는 일
C-1	AWS 쪽만 사라집니다. <code>modules/</code> 는 그대로 남습니다

오늘 나와야 하는 숫자

이 표와 다르면 멈추고 물어보세요. 넘어가면 뒤에서 더 크게 터집니다.

지점	나와야 하는 값	다르면
A-1 버킷 확인	OK: 버킷 있음 과 ACTIVE	week3 bootstrap 확인
A-4 <code>init</code>	- <code>network</code> in <code>../../modules/network</code> 한 줄. <code>compute</code> 는 아직 없습니다	<code>source</code> 경로 확인
A-4 <code>init</code>	Terraform has been successfully initialized!	버킷 이름 확인
B-2 <code>init</code>	- <code>compute</code> in <code>../../modules/compute</code> 가 새로 나옴	<code>source</code> 경로 확인
A-5 <code>validate</code> (<code>extra.tf</code> 있을 때)	Error: Unsupported attribute	일부러 내는 오류입니다
A-5 <code>validate</code> (<code>extra.tf</code> 지운 뒤)	Success! The configuration is valid.	멈춤. TODO ②③ 확인
B-4 <code>validate</code>	Success! The configuration is valid.	TODO ②~⑥ 확인
B-4 <code>plan</code>	Plan: 9 to add, 0 to change, 0 to destroy.	즉시 멈춤. 아래 WARNING
B-5 <code>state list</code>	10줄 (리소스 9 + 데이터 소스 1)	멈춤
B-4 <code>plan</code> 의 <code>ingress</code>	(sensitive value) 로 가려짐	<code>my_ip</code> 의 <code>sensitive</code> 확인
B-5 서버넷 주소	<code>module.network.aws_subnet.public["ap-northeast-2a"]</code>	접두사가 없으면 모듈 밖에 만든 것입니다
B-5 S3 객체	<code>week05/prod/terraform.tfstate</code> 1개	<code>backend.tf</code> 의 <code>key</code> 확인
C-1 <code>destroy</code>	Destroy complete! Resources: 9 destroyed.	멈춤
C-2 <code>state list</code>	빈 출력	멈춤

⚠ 주의

B-4의 `9 to add` 가 오늘의 안전 게이트입니다. `backend.tf` 의 `key` 를 지난주 값인 `week04/app/...` 으로 잘못 두면 week5 스택이 week4 의 state 를 인수합니다. 그대로 apply 하면 지난주 리소스를 지우거나 교체합니다.

숫자가 9가 아니면 apply 하지 말고 멈추세요. `backend` 블록에는 변수도 검사도 넣을 수 없어서 코드로 막을 방법이 없습니다. 이 확인이 유일한 방어선입니다. 개념워크북 13번.

⚠ 주의

week4에서는 `state list` 가 11줄이었습니다. 오늘은 10줄입니다. 리소스 9개는 그대로이고 데이터 소스가 2개에서 1개로 줄었습니다. week4의 `data.aws_availability_zones` 가 오늘 코드에는 없습니다. 숫자만 보고 넘기지 말고 어느 명령의 출력인지 같이 보세요.

Block A. 워크스루 (18분): network 모듈을 만들고 절반만 조립합니다

오늘은 모듈 하나를 먼저 완성하고 그것만 조립해 `plan` 까지 확인합니다. 두 모듈을 한꺼번에 만들면 오류가 났을 때 어느 쪽 문제인지 가릴 수 없습니다.

[확인] A-1. 사전 점검 (2분)

지금 하는 일은 도구와 week3의 저장소가 살아 있는지 보는 것입니다. `{본인-github-id}` 자리는 셸이 채워주지 않으니 week3에서 쓴 `project_name` 을 직접 넣으세요.

```
# pwd == week5-modules
terraform version          # v1.15.8 이상
aws sts get-caller-identity # 계정번호와 사용자 확인

aws s3api head-bucket --bucket "boaz26-w3-{본인-github-id}-tfstate" >/dev/null 2>&1 \
  && echo "OK: 버킷 있음"
aws dynamodb describe-table --region ap-northeast-2 \
  --table-name "boaz26-w3-{본인-github-id}-tflock" --query 'Table.TableStatus' --output text
```

OK: 버킷 있음
ACTIVE

! 꼭 기억

둘 중 하나라도 실패하면 여기서 멈추고 강사에게 알려세요. week3의 `bootstrap` 스택을 다시 apply 해야 합니다. 그때 `project_name` 은 week3에서 쓴 값과 글자 하나까지 같아야 합니다. 버킷 이름이 거기에 걸려 있습니다.

출력된 계정번호 12자리는 화면 공유나 스크린샷에 넣지 마세요.

폴더 구조도 눈으로 확인합니다. 오늘 파일을 여는 곳이 셋입니다.

```
# pwd == week5-modules
find practice -name '*.tf' | sort
```

```
practice/envs/prod/backend.tf
practice/envs/prod/locals.tf
practice/envs/prod/main.tf
practice/envs/prod/outputs.tf
practice/envs/prod/providers.tf
practice/envs/prod/variables.tf
practice/envs/prod/versions.tf
practice/modules/compute/main.tf
practice/modules/compute/outputs.tf
practice/modules/compute/variables.tf
practice/modules/network/main.tf
practice/modules/network/outputs.tf
practice/modules/network/variables.tf
```

`modules/` 쪽에 `providers.tf` 와 `versions.tf` 가 없는 것을 보세요. 모듈은 provider 를 갖지 않습니다. 개념워크북 5번.

[실습] A-2. TODO ①: week3 버킷을 이어 쓰고 값 채우기 (3분)

지금 하는 일은 오늘 만들 것의 state 를 week3 버킷의 새 `key` 에 저장하는 것입니다.

```
# pwd == practice/envs/prod
cd practice/envs/prod
cp example.tfvars terraform.tfvars
curl -4 ifconfig.me      # 내 공인 IP 확인
```

`terraform.tfvars` 를 열어 두 줄을 본인 값으로 수정합니다. `region` 은 그대로 유지합니다.

```
region      = "ap-northeast-2"
project_name = "boaz26-w5-kdh1834"
my_ip       = "1.2.3.4"
```

⚠ 주의

`my_ip` 에 `/32` 를 붙이지 마세요. compute 모듈이 `"${var.my_ip}/32"` 로 구성하기 때문에, 이미 붙여두면 `1.2.3.4/32/32` 가 되어 시큐리티 그룹 생성이 실패합니다. `terraform.tfvars` 는 커밋 금지입니다. 본인 공인 IP가 들어 있습니다.

`vpc_cidr` 과 `subnet_cidrs` 는 `variables.tf` 에 `default` 가 있으니 입력하지 않아도 됩니다. 기본값은 이렇습니다. 맵의 `key` 가 AZ 이름이고 값이 CIDR 입니다.

```
subnet_cidrs = {
  "ap-northeast-2a" = "10.0.1.0/24"
  "ap-northeast-2c" = "10.0.2.0/24"
}
```

이제 `backend.tf` 를 열어 `CHANGE-ME` 두 곳을 `week3` 값으로 수정합니다. 방금 정한 `project_name` 과 다른 값입니다.

```

terraform {
  backend "s3" {
    bucket      = "boaz26-w3-kdh1834-tfstate"
    key         = "week05/prod/terraform.tfstate"
    region     = "ap-northeast-2"
    dynamodb_table = "boaz26-w3-kdh1834-tflock"
    encrypt     = true
  }
}

```

⚠ 주의

`key` 는 이미 채워져 있습니다. 건드리지 마세요. week3 리포에서 `terraform output -raw backend_config` 를 복사해 오면 `key` 가 `week03/app/terraform.tfstate` 로 나오고, week4 코드에서 복사해 오면 `week04/app/...` 으로 나옵니다. 그 줄까지 붙여넣으면 지난 주 state 를 인수합니다.

`key` 의 `prod` 는 이 폴더가 `envs/prod` 라서 붙었습니다. 개념워크북 13번.

[실습] A-3. TODO ②: network 모듈을 채웁니다 (7분)

지금 하는 일은 week4의 `network.tf` 를 모듈의 변수로 매개변수화하는 것입니다.

먼저 계약을 읽습니다. 이 두 파일은 고치지 않습니다.

```

# pwd == practice/envs/prod
cat ../../modules/network/variables.tf
cat ../../modules/network/outputs.tf

```

```

variable "vpc_cidr" {
  description = "이 모듈이 만들 VPC 의 CIDR 블록"
  type        = string
}
variable "subnet_cidrs" {
  description = "퍼블릭 서브넷. key 가 AZ 이름, 값이 CIDR 입니다"
  type        = map(string)
}
variable "name_prefix" {
  description = "Name 태그 앞에 붙일 접두어. 루트가 넘겨줍니다"
  type        = string
}
variable "tags" {
  description = "공통 태그. 모듈은 Name 만 merge 로 엮습니다"
  type        = map(string)
  default     = {}
}

```

```

output "vpc_id" {
  description = "만들어진 VPC 의 ID. compute 모듈로 넘깁니다"
  value       = aws_vpc.this.id
}

```

```

output "subnet_ids" {
  description = "AZ 이름을 key 로 하는 서브넷 ID 맵"
  value       = { for k, s in aws_subnet.public : k => s.id }
}
output "public_subnet_id" {
  description = "모듈이 대신 골라 내주는 서브넷. 개념워크북 9번"
  value       = values(aws_subnet.public)[0].id
}

```

output 이 `aws_vpc.this` 와 `aws_subnet.public` 이라는 이름을 요구하고 있습니다. `main.tf` 에서 리소스 이름을 다르게 지으면 output 이 그 리소스를 찾을 수 없습니다. 이름이 계약의 일부라는 뜻입니다.

`modules/network/main.tf` 를 열어 리소스 다섯 개를 채웁니다.

```

resource "aws_vpc" "this" {
  cidr_block           = var.vpc_cidr
  enable_dns_hostnames = true
  tags                 = merge(var.tags, { Name = "${var.name_prefix}-vpc" })
}
resource "aws_internet_gateway" "this" {
  vpc_id = aws_vpc.this.id
  tags   = merge(var.tags, { Name = "${var.name_prefix}-igw" })
}
resource "aws_subnet" "public" {
  for_each           = var.subnet_cidrs
  vpc_id             = aws_vpc.this.id
  availability_zone  = each.key
  cidr_block         = each.value
  map_public_ip_on_launch = true
  tags               = merge(var.tags, { Name = "${var.name_prefix}-${each.key}" })
}
resource "aws_route_table" "public" {
  vpc_id = aws_vpc.this.id
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = aws_internet_gateway.this.id
  }
  tags = merge(var.tags, { Name = "${var.name_prefix}-rt" })
}
resource "aws_route_table_association" "public" {
  for_each = aws_subnet.public
  subnet_id = each.value.id
  route_table_id = aws_route_table.public.id
}

```

week4와 달라진 부분만 설명합니다.

부분	week4	오늘
CIDR 의 출처	<code>var.vpc_cidr</code> (루트의 변수)	<code>var.vpc_cidr</code> (모듈의 변수. 이름은 같고 다른 변수입니다)
이름 접두어	<code>local.name_prefix</code>	<code>var.name_prefix</code> (<code>locals</code> 는 폴더 경계를 넘지 않습니다)
태그	<code>default_tags</code> 가 자동으로 적용	<code>merge(var.tags, { Name = ... })</code> 를 리소스마다 지정

부분	week4	오늘
AZ	<code>each.value.az</code>	<code>each.key</code> (맵의 key 가 AZ 이름입니다)
CIDR	<code>each.value.cidr</code>	<code>each.value</code> (맵의 값이 CIDR 입니다)

💡 팁

`aws_route_table_association` 의 `for_each` 에 `var.subnet_cidrs` 대신 `aws_subnet.public` 을 넣었습니다. `for_each` 로 만든 리소스는 그 자체가 맵이라 다시 순회할 수 있고, key 가 그대로 유지되면서 `each.value` 가 서버넷 객체가 됩니다. week4 개념워크북 12번.

`modules/` 안에서는 `init` 도 `validate` 도 실행하지 않습니다. 확인은 A-4에서 `envs/prod` 를 통해 진행합니다.

[실습] A-4. TODO ③: `envs/prod` 에서 network 모듈만 호출하고 `init` (4분)

지금 하는 일은 모듈 하나만 구성해서 여기까지가 맞는지 검증하는 것입니다.

`envs/prod/main.tf` 를 열어 **network** 블록만 적습니다. compute 는 B-2에서 붙입니다.

```
module "network" {
  source      = "../../modules/network"
  vpc_cidr    = var.vpc_cidr
  subnet_cidrs = var.subnet_cidrs
  name_prefix = local.name_prefix
  tags        = local.common_tags
}
```

왼쪽은 모듈의 변수 이름이고 오른쪽은 루트의 값입니다. `name_prefix` 와 `tags` 는 `locals.tf` 에서 정의한 값을 전달합니다.

```
# pwd == practice/envs/prod
terraform init
```

```
Initializing the backend...
Initializing modules...
- network in ../../modules/network

Initializing provider plugins...
...
Terraform has been successfully initialized!
```

`- network in ../../modules/network` 한 줄이 출력되어야 합니다. compute 는 아직 호출하지 않았으니 목록에 없습니다.

[함정] `Error: Failed to get existing workspaces: S3 bucket "..."` 가 나오면 버킷 이름이 틀렸거나 week3의 bootstrap 이 없는 것입니다. A-1로 돌아가세요.

[함정] `dynamodb_table` 이 deprecated 라는 경고가 함께 뜹니다. week3·week4에서 본 그 경고이고 정상입니다.

여기까지가 맞는지 `plan` 으로 확인합니다. 아직 apply 하지 마세요.

```
# pwd == practice/envs/prod
terraform plan
```

```
Plan: 7 to add, 0 to change, 0 to destroy.
```

7은 network 모듈이 만드는 것입니다. VPC 1 + IGW 1 + 서브넷 2 + 라우트 테이블 1 + 연결 2 입니다. 여기에 B-1에서 시큐리티 그룹과 EC2 가 더해져 9가 됩니다.

⚠ 주의

7이 아니면 멈추세요. 5가 나오면 `for_each` 가 안 걸려서 서브넷과 연결이 하나씩만 잡힌 것이고, 삭제나 교체가 섞여 있으면 `backend.tf` 의 `key` 가 지난 주 값일 수 있습니다.

[함정] [관찰 🍷] A-5. 모듈 안은 밖에서 보이지 않습니다 (2분)

지금 하는 일은 캡슐화를 오류로 검증하는 것입니다. 의도적으로 잘못된 코드를 작성합니다.

라우트 테이블 ID 를 루트에서 참조해보려고 합니다. 임시 파일을 생성하세요.

```
# pwd == practice/envs/prod
cat > extra.tf <<'EOF'
output "rt_id" {
  value = module.network.aws_route_table.public.id
}
EOF
terraform validate
```

```
Error: Unsupported attribute
```

```
on extra.tf line 2, in output "rt_id":
 2:   value = module.network.aws_route_table.public.id
   |_____
   | module.network is object with 3 attributes
```

```
This object does not have an attribute named "aws_route_table".
```

`module.network is object with 3 attributes` 줄을 보세요. 모듈 안에 리소스가 일곱 개 있는데 밖에서 보이는 것은 output 세 개뿐입니다. 모듈 내부 리소스의 주소를 모듈 외부에서 참조하는 문법은 없습니다. 필요하면 `outputs.tf` 에 output 을 추가해야 하고, 그것이 계약을 수정하는 일입니다.

확인이 끝나면 임시 파일을 지우세요.

```
# pwd == practice/envs/prod
rm extra.tf
terraform validate
```

Success! The configuration is valid.

여기서 **Success!** 가 나오는 것이 정상입니다. compute 모듈은 아직 부르지 않았고 `envs/prod/outputs.tf` 의 TODO ⑥도 비어 있으니, 지금 검사 대상은 network 모듈까지입니다.

참고

`validate` 는 지금 코드에서 참조가 맞물리지만 붐뮈. 부르지 않은 모듈은 검사하지 않습니다.
`modules/compute/main.tf` 가 비어 있고 그 폴더의 `outputs.tf` 가 `aws_instance.web` 을 가리키고 있어도, compute 모듈을 부르기 전까지는 오류가 나지 않습니다. 그 오류는 B-2에서 모듈을 붙인 뒤에 드러납니다.

A-5 기록

- `plan` 이 센 리소스 개수 (network 만): _____
- `module.network is object with ___ attributes` 의 숫자: _____
- 그 숫자가 무엇의 개수인가: _____
- 모듈 안 리소스를 밖에서 쓰려면 무엇을 해야 하는가: _____

Block B. 각자 진행 (20분): compute 를 붙이고 조립을 마칩니다

강사는 순회합니다. 막히면 `solution/` 을 보되, 먼저 오류 메시지의 첫 줄을 읽어보세요.

[실습] B-1. TODO ④: compute 모듈을 채웁니다 (6분)

지금 하는 일은 week4의 `compute.tf` 를 모듈의 변수로 매개변수화하는 것입니다.

계약을 먼저 읽습니다.

```
# pwd == practice/envs/prod
cat ../../modules/compute/variables.tf
cat ../../modules/compute/outputs.tf
```

`vpc_id` 와 `subnet_id` 를 받는 것에 주목하세요. compute 모듈은 자신이 어느 VPC 의 어느 서브넷에 위치하는지 알지 못한 채 ID 두 개만 받습니다. `instance_type` 은 `default = "t3.micro"` 가 있어서 전달하지 않아도 됩니다.

`modules/compute/main.tf` 를 채웁니다.

```
data "aws_ami" "al2023" {
  most_recent = true
  owners      = ["amazon"]
}
```

```

filter {
  name    = "name"
  values  = ["al2023-ami-2023.*-x86_64"]
}
}
resource "aws_security_group" "web" {
  name      = "${var.name_prefix}-web-sg"
  vpc_id    = var.vpc_id
  ingress {
    from_port = 22
    to_port   = 22
    protocol  = "tcp"
    cidr_blocks = ["${var.my_ip}/32"]
  }
  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
  tags = merge(var.tags, { Name = "${var.name_prefix}-web-sg" })
}
resource "aws_instance" "web" {
  ami                = data.aws_ami.al2023.id
  instance_type      = var.instance_type
  subnet_id          = var.subnet_id
  vpc_security_group_ids = [aws_security_group.web.id]
  root_block_device {
    volume_type      = "gp3"
    volume_size      = 8
    delete_on_termination = true
  }
  metadata_options {
    http_tokens = "required"
  }
  tags = merge(var.tags, { Name = "${var.name_prefix}-web" })
}

```

데이터 소스가 모듈 내부에 위치하는 것을 확인하세요. AMI 조회는 EC2 를 생성하는 작업에 속하므로 compute 모듈의 책임입니다. 이 데이터 소스의 주소도 접두사가 붙어서 `module.compute.data.aws_ami.al2023` 이 됩니다.

`root_block_device` 와 `metadata_options` 는 week3·week4 에서 쓰던 것을 그대로 가져왔습니다. 앞의 것은 루트 볼륨을 gp3 8GiB 로 고정해서 이 문서의 비용 계산이 코드와 어긋나지 않게 합니다. 뒤의 것은 IMDSv2 를 강제하며, AL2023 의 기본값이지만 의도를 코드에 남깁니다. 둘 다 `resource` 안의 블록이라 리소스 개수는 늘지 않습니다. `plan` 은 그대로 `9 to add` 입니다.

⚠ 주의

이름 패턴을 `al2023-ami-2023.*-x86_64` 로, `2023.` 까지 고정해서 적으세요. `al2023-ami.*-x86_64` 로 느슨하게 두면 ECS 전용 이미지와 minimal 이미지가 후보에 들어오고 `most_recent = true` 가 그중 최신을 골라버립니다. week3 개념워크북 15번에서 다룬 함정입니다.

`owners = ["amazon"]` 은 이것을 막아주지 못합니다. ECS 전용 AMI 를 발행하는 계정도 소유자 별칭이 `amazon` 입니다. 서울 리전에서 두 패턴이 고르는 이미지가 이렇게 같립니다.

패턴	콜라지는 이미지	루트 볼륨
al2023-ami-2023.*-x86_64	al2023-ami-2023.12.*-kernel-6.12-x86_64	gp3 8GiB
al2023-ami-*x86_64	al2023-ami-ecs-hvm-2023.*-kernel-6.1-x86_64	gp3 30GiB

루트 볼륨이 30GiB 가 되면 이 문서의 비용 계산이 어긋납니다. EBS 가 월 \$0.73 에서 \$2.74 로 늘어서 한 달 방치 금액이 \$13.87 이 아니라 약 \$15.88 이 됩니다. `plan` 은 그대로 `9 to add` 라서 안전 게이트에 걸리지 않습니다.

🚫 절대 금지

EC2 는 1대만 유지합니다. `aws_instance.web` 에 `for_each` 나 `count` 를 붙이지 마세요. 요금이 대수만큼 곱해집니다.

[실습] [함정] B-2. TODO ⑤: compute 를 조립하고 `init` 을 다시 (3분)

지금 하는 일은 두 모듈을 연결하고, `init` 을 다시 실행해야 하는 이유를 확인하는 것입니다.

`envs/prod/main.tf` 에 `compute` 블록을 더합니다.

```
module "compute" {
  source      = "../modules/compute"
  vpc_id      = module.network.vpc_id
  subnet_id   = module.network.public_subnet_id
  my_ip       = var.my_ip
  name_prefix = local.name_prefix
  tags        = local.common_tags
}
```

가운데 두 줄이 오늘의 핵심입니다. `network` 의 `output` 을 `compute` 의 변수로 전달하는 부분이고, 두 모듈이 만나는 유일한 지점입니다.

`init` 을 돌리지 말고 먼저 `plan` 을 해보세요.

```
# pwd == practice/envs/prod
terraform plan
```

Error: Module not installed

```
on main.tf line 8:
  8: module "compute" {
```

This module is not yet installed. Run "terraform init" to install all modules required by this configuration.

줄번호는 본인 `main.tf` 에 남아 있는 주석 분량에 따라 달라집니다. `8` 이 아니어도 `module "compute" {` 를 가리키면 같은 상황입니다.

오류가 새로 추가한 블록을 가리킵니다. `init` 은 provider 를 받아오면서 자식 모듈의 위치도 함께 기록합니다. `module` 블록을 추가하거나 `source` 를 고치면 반드시 다시 실행해야 합니다. 개념워크북 11번.

```
# pwd == practice/envs/prod
terraform init
```

```
Initializing modules...
- compute in ../../modules/compute
```

network 는 이미 등록되어 있으니 compute 한 줄만 새로 나옵니다.

💡 팁

모듈 안의 `main.tf` 내용만 고친 경우에는 `init` 을 다시 돌릴 필요가 없습니다. 위치가 바뀐 것이 아니기 때문입니다. 다시 돌려야 하는 것은 `module` 블록을 추가·삭제했거나 `source` 를 바꿨을 때입니다.

[실습] B-3. TODO ⑥: 모듈 output 을 루트에서 다시 냅니다 (2분)

지금 하는 일은 모듈이 내보낸 값을 루트 모듈 밖으로 다시 출력하는 것입니다.

`envs/prod/outputs.tf` 를 채웁니다.

```
output "vpc_id" { value = module.network.vpc_id }
output "subnet_ids" { value = module.network.subnet_ids }
output "instance_public_ip" { value = module.compute.public_ip }
```

output 을 두 번 정의하는 것이 중복처럼 보일 수 있습니다. 하지만 계층이 다릅니다. 모듈의 `outputs.tf` 는 **모듈 외부로** 값을 전달하는 정의이고, 루트의 `outputs.tf` 는 **terraform output 명령과 사용자에게** 값을 제공하는 정의입니다. 모듈이 내보낸 값을 루트가 다시 출력하지 않으면 `terraform output` 에 아무 값도 표시되지 않습니다.

[확인] B-4. `fmt` → `validate` → `plan` (2분)

지금 하는 일은 apply 전에 세 단계로 확인하는 것입니다.

```
# pwd == practice/envs/prod
terraform fmt -recursive ../../..
terraform validate
terraform plan
```

```
Success! The configuration is valid.
```

```
Plan: 9 to add, 0 to change, 0 to destroy.
```

`fmt` 에 `-recursive ../../` 를 준 것은 모듈 폴더까지 함께 정리하기 위해서입니다. `envs/prod` 에서 `terraform fmt` 만 치면 이 폴더의 파일만 봅니다.

⚠️ 주의

9가 아니면 apply 하지 말고 멈추세요. 70이면 compute 모듈 블록이 안 붙은 것이고, 다른 숫자가 나오거나 삭제·교체가 섞여 있으면 `backend.tf` 의 `key` 가 지난 주 것일 수 있습니다. 그대로 apply 하면 지난 주 리소스를 건드립니다.

plan 출력에서 주소를 미리 봅니다. 모듈 경로 접두사가 붙어 있습니다.

```
# module.network.aws_vpc.this will be created
# module.compute.aws_instance.web will be created
```

💡 팁

시큐리티 그룹 쪽을 보세요. `ingress` 블록이 (`sensitive value`) 로 가려져 있습니다. `my_ip` 에 붙은 `sensitive` 가 모듈 경계를 넘어 compute 모듈까지 따라온 결과입니다. 이 화면은 캡처해도 공인 IP 가 남지 않습니다. 표시를 직접 붙여보는 것은 B-6입니다.

[확인] [관찰 🏎️] B-5. apply 와 모듈 경로 접두사 (5분)

지금 하는 일은 리소스를 실제로 생성하고, 주소에 접두사가 붙은 것을 확인하는 것입니다.

```
# pwd == practice/envs/prod
terraform apply      # yes
terraform state list | tee ~/week5-state-list.txt
```

`tee` 로 홈 디렉터리에 사본을 남기는 것에 주의하세요. C-1 에서 `destroy` 를 하면 `state list` 가 비어서 다시 출력할 수 없고, C-3 의 마스킹과 제출물이 이 파일을 씁니다. 리포 안이 아니라 ~ 에 두는 것은 마스킹 전에 실수로 커밋되지 않게 하기 위해서입니다.

Apply complete! Resources: 9 added, 0 changed, 0 destroyed.

`state list` 가 10줄입니다. 데이터 소스 한 줄은 읽기라서 `9 to add` 에는 들어가지 않습니다.

```
module.compute.data.aws_ami.al2023
module.compute.aws_instance.web
module.compute.aws_security_group.web
module.network.aws_internet_gateway.this
module.network.aws_route_table.public
module.network.aws_route_table_association.public["ap-northeast-2a"]
module.network.aws_route_table_association.public["ap-northeast-2c"]
module.network.aws_subnet.public["ap-northeast-2a"]
module.network.aws_subnet.public["ap-northeast-2c"]
module.network.aws_vpc.this
```

모든 줄이 `module.` 로 시작합니다. `envs/prod` 가 직접 생성한 리소스는 하나도 없고, 모듈 구성만 수행했기 때문입니다.
output 을 봅니다.

```
# pwd == practice/envs/prod
terraform output
```

```
instance_public_ip = "3.35.x.x"
subnet_ids = {
  "ap-northeast-2a" = "subnet-..."
  "ap-northeast-2c" = "subnet-..."
}
vpc_id = "vpc-..."
```

주소 하나를 열어봅니다.

```
# pwd == practice/envs/prod
terraform state show 'module.network.aws_subnet.public["ap-northeast-2a"]'
```

⚠ 주의

작은따옴표로 감싸야 합니다. 안 감싸면 셸이 대괄호와 큰따옴표를 먼저 해석해서 `zsh: no matches found` 가 납니다.
week4와 같습니다.

S3 객체도 확인합니다.

```
# pwd == practice/envs/prod
aws s3 ls "s3://boaz26-w3-{본인-github-id}-tfstate/week05/prod/"
```

week3·week4 의 객체가 같은 버킷에 그대로 남아 있는지도 보세요. `key` 가 다르면 서로 다른 state 입니다.

```
# pwd == practice/envs/prod
aws s3 ls --recursive "s3://boaz26-w3-{본인-github-id}-tfstate/"
```

B-5 기록

- `plan` 마지막 줄: `Plan: ___ to add, ___ to change, ___ to destroy.`
- `state list` 줄 수: _____ (리소스 _____ + 데이터 소스 _____)
- `module.network.` 로 시작하는 줄 수: _____ / `module.compute.` 로 시작하는 줄 수: _____
- 서브넷 주소 두 줄을 그대로 옮겨 적기: _____
- `terraform output subnet_ids` 가 찍은 key 두 개: _____
- 버킷 안에 있는 `.tfstate` 객체 key 를 전부 적기: _____
- week4의 주소와 비교해 무엇이 달라졌는지 한 문장: _____

[실습] [관찰 📖] B-6 (심화 ⭐). `sensitive` 가 막는 것과 고르는 근거 (2분)

지금 하는 일은 두 가지입니다. `sensitive` 의 경계를 확인하고, 서브넷 선택 기준을 루트 모듈로 옮깁니다.

첫째, `sensitive` 가 어디까지 막는지 봅니다. `my_ip` 에는 표시가 두 곳에 붙어 있습니다.

```
# pwd == practice/envs/prod
grep -A5 'variable "my_ip"' variables.tf ../../modules/compute/variables.tf
```

루트에만 표시를 지정해도 값이 모듈로 전달될 때 표시가 함께 전달됩니다. 모듈 쪽에도 표시를 지정하는 것은 이 모듈을 다른 곳에서 사용할 때 호출하는 쪽이 표시 지정을 잊을 수 있는 경우를 대비한 것입니다.

state 에서는 어떤지 확인합니다.

```
# pwd == practice/envs/prod
terraform state pull | grep -o '"cidr_blocks":\[^\]*]' | head
```

본인 공인 IP 가 평문으로 표시됩니다. `sensitive` 가 변경하는 것은 CLI 출력이고, 저장소 보안은 week3에서 설정한 버킷 암호화와 퍼블릭 접근 차단이 담당합니다. week4 개념워크북 6번.

둘째, EC2 의 서브넷 배치 결정을 루트 모듈로 옮깁니다. `envs/prod/main.tf` 의 compute 블록에서 한 줄을 수정하세요.

```
subnet_id = module.network.subnet_ids["ap-northeast-2a"]
```

```
# pwd == practice/envs/prod
terraform plan
```

```
No changes. Your infrastructure matches the configuration.
```

지금은 `public_subnet_id` 가 참조하던 것과 같은 서브넷이라 아무 변화가 없습니다. 달라진 점은 어느 서브넷을 사용하는지가 루트 모듈 파일에 명시된다는 것입니다. `public_subnet_id` 는 `values(...)[0]` 이라서 사전순 첫 key 를 제거하면 참조 대상이 아무 경고 없이 바뀌고, 그러면 EC2 가 교체됩니다. 개념워크북 9번.

맵에 없는 key 를 입력하면 어떻게 되는지도 확인하세요. 방금 수정한 줄의 key 를 `"ap-northeast-2b"` 로 변경하고 `plan` 을 실행합니다.

```
# pwd == practice/envs/prod
terraform plan
```

```
Error: Invalid index
```

```
on main.tf line 11, in module "compute":
11:   subnet_id = module.network.subnet_ids["ap-northeast-2b"]
    |_____
    | module.network.subnet_ids is object with 2 attributes
```

```
The given key does not identify an element in this collection value.
```

`module.network.subnet_ids` is object with 2 attributes 줄을 보세요. A-5 의 `module.network` is object with 3 attributes 와 같은 모양입니다. 맵에 key 가 둘뿐이라는 것을 Terraform 이 `plan` 단계에서 알기 때문에 apply 까지 가지 않고 plan 에서 오류가 납니다.

`public_subnet_id` 를 쓸 때는 이런 오류가 나지 않습니다. 모듈이 자동으로 하나를 선택하기 때문입니다. 오류가 나지 않는 쪽이 편하지만, 무엇을 선택하고 있는지도 보이지 않습니다.

확인이 끝나면 key 를 `"ap-northeast-2a"` 로 복원하세요.

B-6 기록

- `sensitive` 표시가 붙어 있는 파일 두 개: _____
- `plan` 에서 `ingress` 가 어떻게 보였는가: _____
- `state pull` 에서 공인 IP 가 평균으로 보였는가: _____
- `sensitive` 가 막는 것과 못 막는 것을 한 문장으로: _____
- `public_subnet_id` 와 `subnet_ids["..."]` 의 차이를 한 문장으로: _____

[실습] B-7 (심화 ★). `envs/dev` 로 모듈을 재사용하기

지금 하는 일은 모듈을 수정하지 않고 환경을 하나 더 생성하는 것입니다. `apply` 는 실행하지 말고 `plan` 까지만 수행하세요. EC2 가 두 대가 됩니다.

```
# pwd == practice/envs/dev
cd ..
mkdir -p dev && cp prod/*.tf prod/example.tfvars dev/
cd dev && cp example.tfvars terraform.tfvars
```

`dev/backend.tf` 의 `key` 를 `week05/dev/terraform.tfstate` 로 바꾸고, `dev/locals.tf` 의 `Env` 를 `dev` 로, `terraform.tfvars` 의 `project_name` 을 dev 용으로 바꿉니다. `dev/main.tf` 의 `source` 는 그대로 `../../modules/...` 입니다. 깊이가 같기 때문입니다.

🚫 절대 금지

`key` 를 먼저 바꾸세요. `cp prod/*.tf dev/` 가 `key = "week05/prod/terraform.tfstate"` 를 그대로 복사해 왔기 때문에, 안 바꾸고 `init` 하면 dev 폴더가 prod 의 state 를 그대로 물려받습니다. 그러면 아래 `plan` 이 `9 to add` 가 아니라 `No changes.` 로 나오고, 그 상태에서 `destroy` 를 하면 `prod` 리소스가 지워집니다.

`plan` 이 `No changes.` 로 나오면 `key` 를 안 바꾼 것입니다. `9 to add` 를 확인하고 넘어가세요.

```
# pwd == practice/envs/dev
terraform init
terraform plan      # 9 to add. 모듈 코드는 한 줄도 고치지 않았습나다
```

확인이 끝나면 `dev` 폴더를 지우세요. apply 하지 않았으므로 AWS 에는 아무것도 없습니다. S3 객체도 생기지 않습니다.

backend 가 객체를 만드는 시점은 state 를 처음 쓰는 `apply` 이고, `init` 과 `plan` 은 읽기와 잠금만 합니다. 로컬 `dev`

폴더만 지우면 됩니다.

Block C. 정리 (7분): destroy 와 잔존 점검

🚫 절대 금지

이 블록을 건너뛰면 EC2가 계속 실행됩니다. 시간당 \$0.0190, 한 달이면 약 \$13.87입니다. 실습의 마지막 순서로 생각하지 말고 독립된 필수 블록으로 여기세요.

지우는 것은 오늘 만든 것뿐입니다. week3의 S3와 DynamoDB는 7주차까지 남깁니다.

[확인] C-1. envs/prod destroy (3분)

destroy 는 루트 모듈에서 한 번만 실행합니다. `modules/` 안에서는 실행하지 않습니다. 그 폴더에는 state 가 없습니다.

```
# pwd == practice/envs/prod
terraform destroy      # yes
```

Destroy complete! Resources: 9 destroyed.

B-7의 심화를 진행했다면 `envs/dev` 도 정리했는지 확인하세요.

[함정] destroy 가 시큐리티 그룹이나 서브넷에서 멈추면 ENI 가 남아 있는 경우입니다. 잠시 기다린 뒤 한 번 더 실행하세요.

[확인] [관찰 🕒] C-2. state 와 계정을 따로 확인 (2분)

`state list` 가 비었다는 것은 Terraform 이 더 이상 관리하지 않는다는 뜻일 뿐, 계정에 리소스가 없다는 뜻이 아닙니다.

```
# pwd == practice/envs/prod
terraform state list      # 빈 출력이어야 합니다
```

계정 쪽은 태그로 조회합니다. `locals.tf` 에서 지정한 `Study` 와 `Env` 를 기준으로 사용합니다.

```
# pwd == practice/envs/prod
R=ap-northeast-2
aws ec2 describe-instances --region $R \
  --filters "Name=tag:Study,Values=boaz-terraform-26" \
    "Name=instance-state-name,Values=pending,running,stopped,stopped" \
  --query 'Reservations[].Instances[].InstanceId' --output text

aws ec2 describe-volumes --region $R \
  --filters "Name=tag:Study,Values=boaz-terraform-26" \
  --query 'Volumes[].VolumeId' --output text
```

```
aws ec2 describe-addresses --region $R --query 'Addresses[].PublicIp' --output text

aws ec2 describe-nat-gateways --region $R \
  --filter "Name=state,Values=pending,available" \
  --query 'NatGateways[].NatGatewayId' --output text
```

네 명령이 모두 빈 출력이어야 합니다. **aws** CLI가 없으면 콘솔에서 EC2 인스턴스 · EBS 볼륨 · Elastic IP · NAT Gateway 네 가지를 서울 리전에서 직접 확인하세요.

인스턴스와 볼륨은 **Env** 가 아니라 **Study** 태그로 조회합니다. B-7 심화에서 **envs/dev** 를 만들면 그쪽은 **Env=dev** 라서, **Env=prod** 로 좁히면 실수로 apply 한 dev 리소스를 놓칩니다.

week3의 저장소는 남아 있어야 정상입니다.

```
# pwd == practice/envs/prod
aws s3 ls | grep tfstate
aws dynamodb list-tables --region ap-northeast-2 --query 'TableNames' --output text
```

🚫 절대 금지

stop 은 destroy 가 아닙니다. 인스턴스를 멈추면 EC2와 퍼블릭 IPv4 요금은 멈추지만 EBS 8GiB의 월 \$0.73은 계속 나갑니다.

C-2 기록

- **terraform state list** 가 비었는가: _____
- EC2 · EBS · Elastic IP · NAT Gateway 조회가 전부 빈 출력인가: _____
- week3의 S3 버킷과 DynamoDB 테이블은 남아 있는가: _____
- 버킷에 남은 **.tfstate** 객체 개수와 key: _____
- **modules/** 폴더를 지우면 AWS의 VPC가 사라지는가, 왜 그런가: _____

[실습] C-3. 제출 전 마스킹 (1분)

지금 하는 일은 증빙에서 숨겨야 할 값을 제거하는 것입니다.

```
# pwd == week5-modules
cd ../../..
ID=본인-github-id          # 예시 값 그대로 두지 마세요
echo "$ID"                  # 바꿨는지 눈으로 확인
mkdir -p "submissions/$ID"
```

B-5에서 **~/week5-state-list.txt** 로 남겨둔 사본을 가져옵니다. C-1에서 destroy 를 했으므로 지금 **state list** 를 다시 출력하면 빈 파일이 됩니다.

계정번호 12자리와 IP 를 숨깁니다. macOS 와 GNU 에서 **sed -i** 문법이 다르므로 새 파일을 생성하는 방식이 안전합니다.

```
# pwd == week5-modules
sed -E \
  -e 's/[0-9]{12}/<account-id>/g' \
  -e 's/([0-9]{1,3}\.){3}[0-9]{1,3}/x.x.x.x/g' \
  ~/week5-state-list.txt > "submissions/$ID/state-list.txt"
```

마스크링이 실제로 됐는지 파일을 눈으로 확인합니다. 10줄이 그대로 있고 IP 와 계정번호만 바뀌어 있어야 합니다.

```
# pwd == week5-modules
cat "submissions/$ID/state-list.txt"
grep -nE '([0-9]{1,3}\.){3}[0-9]{1,3}|[0-9]{12}' "submissions/$ID/state-list.txt"
```

두 번째 명령은 아무것도 안 나와야 정상입니다. 커밋 직전 검사는 스테이징한 뒤에 해야 의미가 있으므로 아래 제출 절차에 넣어두었습니다.

🚫 절대 금지

스크린샷도 마스크하세요. 명령으로는 감지되지 않습니다. `sensitive` 는 plan 의 `ingress` 를 숨기지만 `terraform.tfvars` 와 state 에는 공인 IP 가 평문으로 저장됩니다. 숨겨야 하는 것은 공인 IP와 계정번호 12자리입니다.

[개념] C-4. week3 저장소는 그대로 둡니다 (1분)

읽고 진행하세요. week3에서 생성한 S3 버킷과 DynamoDB 테이블은 7주차까지 유지하고, 6주차도 같은 버킷에 `key` 만 변경하여 사용합니다. 삭제 절차는 week3 실습워크북 C-5에 있고 이번 주에는 실행하지 않습니다.

`modules/` 폴더도 삭제하지 마세요. 과제에서 그대로 계속 사용합니다.

세션 후: 실습 결과 제출 (브랜치 → PR)

세션 중에는 `practice/` 에서 실습하면 됩니다. 세션이 끝난 뒤 아래 절차를 진행하세요.

```
# pwd == week5-modules
ID=본인-github-id # {github-id} 는 셸이 치환해주지 않습니다

git switch main && git pull
git switch -c "week5/$ID"

mkdir -p "submissions/$ID"
cp -R practice/modules "submissions/$ID/"
mkdir -p "submissions/$ID/envs/prod"
cp practice/envs/prod/*.tf "submissions/$ID/envs/prod/"
cp practice/envs/prod/example.tfvars "submissions/$ID/envs/prod/"
# C-3에서 저장한 state-list.txt 는 이미 그 폴더에 있습니다 (마스크 확인!)
# 관찰 🍷 답안을 submissions/$ID/observations.md 에 정리

git add "submissions/$ID"
```

```
# 스테이징한 내용에서 공인 IP · 계정번호 · 액세스 키를 찾습니다. 빈 출력이어야 합니다.
git diff --cached | grep -nE '([0-9]{1,3}\.){3}[0-9]{1,3}|[0-9]{12}|AKIA[0-9A-Z]{16}'
git status --porcelain | grep -E 'terraform\.tfvars|\.tfstate|\.pem'
git status # tfvars / tfstate / .terraform 안 올라갔는지!
git commit -m "week5: $ID 실습 제출"
git push -u origin "week5/$ID"
```

넣을 것

modules/ 의 .tf 파일 전부

envs/prod 의 .tf 파일 (backend.tf 포함)

example.tfvars (값은 채우지 않은 상태)

state-list.txt (마스킹 완료. B-5의 10줄)

observations.md (관찰 🐞 답안)

넣지 말 것

terraform.tfvars

terraform.tfstate · *.backup

.terraform/ 폴더

.pem 파일

공인 IP · 계정번호 12자리

복사가 끝나면 practice/ 는 되돌려 둡니다.

```
# pwd == week5-modules
git restore practice/ # 복사를 끝낸 뒤에만! 스켈레톤으로 되돌아갑니다
rm -rf practice/envs/dev # B-7 심화를 했다면
```

그다음 GitHub에서 week5/{github-id} 에서 main 으로 PR을 올리면 강사가 리뷰 후 머지합니다.

! 꼭 기억

제출은 submissions/{github-id}/ 본인 폴더에 하세요. practice/ 를 직접 고쳐서 올리면 머지하는 순간 다음 사람이 풀 # TODO 가 사라집니다.

과제③은 이 리포가 아니라 assignments 리포의 round3-week5/{github-id}/ 에 냅니다. 명세는 assignment/ASSIGNMENT.md 에 있습니다.

체크포인트 (Definition of Done)

- [] A-1: week3 버킷과 테이블이 살아 있는 것 확인
- [] A-2: backend.tf 의 key 가 week05/prod/terraform.tfstate
- [] A-3: modules/network/main.tf 에 리소스 5개. provider 블록은 넣지 않았습니다
- [] A-4: init 에 - network in ../../modules/network 가 나옴
- [] A-4: network 만으로 plan 이 7 to add
- [] A-5: Unsupported attribute 로 캡슐화 확인. extra.tf 는 지웠습니다
- [] B-1: modules/compute/main.tf 에 데이터 소스 1개 + 리소스 2개. EC2 는 1대
- [] B-2: Module not installed 를 본 뒤 init 재실행

- [] B-3: 모듈 output 이 루트 output 으로 재노출됨
- [] B-4: `validate` 성공. `plan` 이 `Plan: 9 to add, 0 to change, 0 to destroy.`
- [] B-4: `plan` 의 `ingress` 가 `(sensitive value)` 로 가려짐
- [] B-5: `state list` 10줄. 모든 줄이 `module.` 로 시작
- [] B-5: 서브넷 주소가 `module.network.aws_subnet.public["ap-northeast-2a"]` 형태
- [] B-5: S3 에 `week05/prod/terraform.tfstate` 객체 1개
- [] C-1: `destroy` 완료 (`9 destroyed`)
- [] C-2: `state list` 빈 출력. 계정 조회 네 개 전부 빈 출력
- [] C-3: 마스킹 확인. 공인 IP 와 계정번호가 증빙에 없습니다
- [] week3의 S3와 DynamoDB는 지우지 않았습니다

오늘의 명령 요약

```
# Block A
cd practice/envs/prod
cp example.tfvars terraform.tfvars      # project_name · my_ip
# backend.tf 의 CHANGE-ME 두 곳을 week3 값으로 (key 는 건드리지 않기)
# TODO ② modules/network/main.tf 채우기
# TODO ③ main.tf 에 module "network" 만
terraform init                          # - network in ../../modules/network
terraform plan                          # Plan: 7 to add
# extra.tf 로 module.network.aws_route_table 참조 -> Unsupported attribute
rm extra.tf

# Block B
# TODO ④ modules/compute/main.tf 채우기
# TODO ⑤ main.tf 에 module "compute" 추가
terraform plan                          # Module not installed <- 일부러 봅니다
terraform init                          # - compute in ../../modules/compute
# TODO ⑥ outputs.tf 채우기
terraform fmt -recursive ../../..
terraform validate                      # Success!
terraform plan                          # Plan: 9 to add <- 아니면 멈춤
terraform apply                         # yes. 9 added
terraform state list                    # 10줄. 전부 module. 로 시작
terraform output
terraform state show 'module.network.aws_subnet.public["ap-northeast-2a"]' # 작은따옴표 필수

# Block C
terraform destroy                       # 9 destroyed
terraform state list                    # 빈 출력
# 계정 조회 네 개 (C-2) 전부 빈 출력
```

다음 주 예고

Week 6. State Drift & 기존 리소스 import **[대면]**. 오늘 만든 주소 체계가 다음 주의 문제가 됩니다.

오늘 본 것	Week6에서 문제가 되는 지점
리소스 주소가 <code>module.network.aws_vpc.this</code> 가 되었습니다	콘솔에서 직접 만든 리소스는 이 주소가 없습니다. state 밖의 실물을 코드로 가져오는 절차가 필요합니다
<code>state list</code> 가 비있어도 계정은 따로 확인했습니다	그 둘이 어긋난 상태에 이름이 있습니다. drift 입니다
<code>plan</code> 이 <code>9 to add</code> 인지를 게이트로 썼습니다	누가 콘솔에서 값을 바꾸면 <code>~</code> 가 뜹니다. 코드를 실물에 맞추지 실물을 코드에 맞추지 정해야 합니다
모듈은 계약만 보이고 안은 안 보입니다	모듈 안 리소스를 <code>import</code> 하려면 <code>module.network.</code> 까지 붙인 주소를 정확히 적어야 합니다
예습: drift 개념, <code>terraform plan</code> 의 <code>~ · + · -</code> 읽는 법, <code>terraform import</code> 문법, <code>import</code> 블록	

부록. 트러블슈팅

증상	원인	해결
<code>Error: Module not installed</code>	<code>module</code> 블록을 추가·삭제했는데 <code>init</code> 을 안 돌림	<code>terraform init</code> 재실행. B-2
<code>Error: Invalid module source address</code>	<code>source</code> 가 <code>./</code> 나 <code>../</code> 로 시작하지 않음	상대경로로 고치기. 메시지가 고칠 형태를 알려줍니다
<code>Error: Unsupported argument</code>	<code>module</code> 블록에 그 모듈에 없는 인자를 넘김	그 모듈의 <code>variables.tf</code> 와 이름 대조
<code>Error: Missing required argument</code>	<code>default</code> 없는 모듈 변수를 안 넘김	오류가 <code>module</code> 첫 줄을 가리킵니다. <code>variables.tf</code> 와 대조
<code>Error: Unsupported attribute (module.x is object with N attributes)</code>	모듈 안 리소스를 밖에서 직접 참조	그 모듈의 <code>outputs.tf</code> 에 output 추가. A-5
<code>Error: Reference to undeclared resource</code> (모듈 안에서)	모듈 안에서 루트의 값이나 다른 모듈을 참조	변수로 받기. 개념워크북 6번
모듈 안에서 <code>local.name_prefix</code> 를 못 찾음	<code>locals</code> 는 폴더 경계를 넘지 않음	<code>var.name_prefix</code> 로 받기
<code>Invalid for_each argument</code>	<code>subnet_cidrs</code> 에 list 를 넣음	<code>map(string)</code> 으로 넣기. week4 개념워크북 12번
<code>Invalid index</code>	<code>subnet_ids["..."]</code> 의 key 가 맵에 없음	<code>terraform output subnet_ids</code> 로 key 확인. B-6
<code>plan</code> 이 7에서 멈춤	compute 모듈 블록이 안 붙음	B-2 확인

증상	원인	해결
<code>plan</code> 이 9가 아닌 이상한 숫자	<code>backend.tf</code> 의 <code>key</code> 가 지난 주 것	apply 하지 말고 <code>key</code> 를 <code>week05/prod/...</code> 로
<code>Failed to get existing workspaces: S3 bucket ... does not exist</code>	week3 버킷 이름 오타 또는 bootstrap 없음	A-1로 돌아가기
<code>Error acquiring the state lock</code>	동시 실행 또는 비정상 종료	아무도 작업 안 하는 것 확인 후 <code>terraform force-unlock <LOCK_ID></code>
<code>Warning: Deprecated Parameter</code>	<code>dynamodb_table</code> 사용	정상입니다. week3 개념워크북 9번
<code>zsh: no matches found</code>	주소의 대괄호를 셸이 해석	작은따옴표로 감싸기
<code>destroy</code> 가 SG/서브넷에서 멈춤	ENI 잔존	잠시 후 재실행

참고 문서

- [Module Blocks](#) : A-4 · B-2
- [Module Sources](#) : `source` 경로 규칙
- [Providers Within Modules](#) : A-3. 모듈에 provider 를 두지 않는 이유
- [Module Outputs](#) : A-5 · B-3
- [values 함수](#) : B-6
- [Refactoring](#) : `moved` 블록. 과제 심화
- week3 실습워크북 B-4 · B-5 : backend 로 state 를 옮기는 절차
- week4 실습워크북 B-1 · B-2 : `for_each` 를 처음 걸었던 자리

이 문서의 출력 중 `Initializing modules...` 와 모듈 두 줄, `Error: Unsupported attribute (object with 3 attributes 포함), Error: Module not installed, Error: Unsupported argument, Error: Missing required argument, Error: Invalid module source address, Success! The configuration is valid.` (A-5 시점과 B-4 시점 각각) 는 `solution/` 코드를 Terraform 1.15.8 · AWS provider 6.61.0 · `darwin_arm64` 에서 실제로 실행해 얻은 값입니다. AWS 자격증명 없이 `terraform init -backend=false` 와 `terraform validate` 로 재현했습니다.

`Plan: 7 to add · Plan: 9 to add · 9 added · 9 destroyed · state list` 10줄은 리소스 정의에서 센 값이고, 실제 apply 로는 확인하지 않았습니다. `state list` 의 출력 정렬 순서와 `terraform output` 의 값 모양도 apply 로 확인하지 않았습니다. B-4의 `(sensitive value)` 와 B-6의 `No changes.` · `Invalid index` · `state pull` 출력도 계산한 값입니다. 비용은 개념워크북 Part 6과 같습니다.

끝났으면 `envs/prod` 를 destroy 했는지, B-7 심화를 했다면 `envs/dev` 도 정리했는지, 그리고 week3의 S3와 DynamoDB는 건드리지 않았는지 한 번 더 확인하세요. EC2가 켜져 있으면 시간당 \$0.0190이 계속 붙고, week3 버킷을 지우면 6주차부터 처음부터 다시 만들어야 합니다.